ARQUITECTURA DE MICROSERVICIOS · SERVICE DISCOVERY · CIRCUIT BREAKER

Taller sobre Patrones de Arquitectura de Microservicios

Estudiantes	Pablo Andrés Guzmán Alarcón — A00399523 Stick Martinez — A0039870 (GitHub: @Stixkl) Alejandro Troya — A00399865 (GitHub: @DavidIsGod)

Tema	Service Discovery con Eureka e implementación de Circuit Breaker con Resilience4j
Repositorios base	`jrquinte/FutureXEurekaServer` · `jrquinte/FutureXCourseApp` · `jrquinte/FutureXCourseCatalog`
Código fuente	**https://github.com/pabloandressoftware/FutureXCourseCatalog/tree/feature/resilience4j** Repositorio `pabloandressoftware/FutureXCourseCatalog`, rama `feature/resilience4j` (pom.xml, application.yml, CatalogController, RestClientConfig)

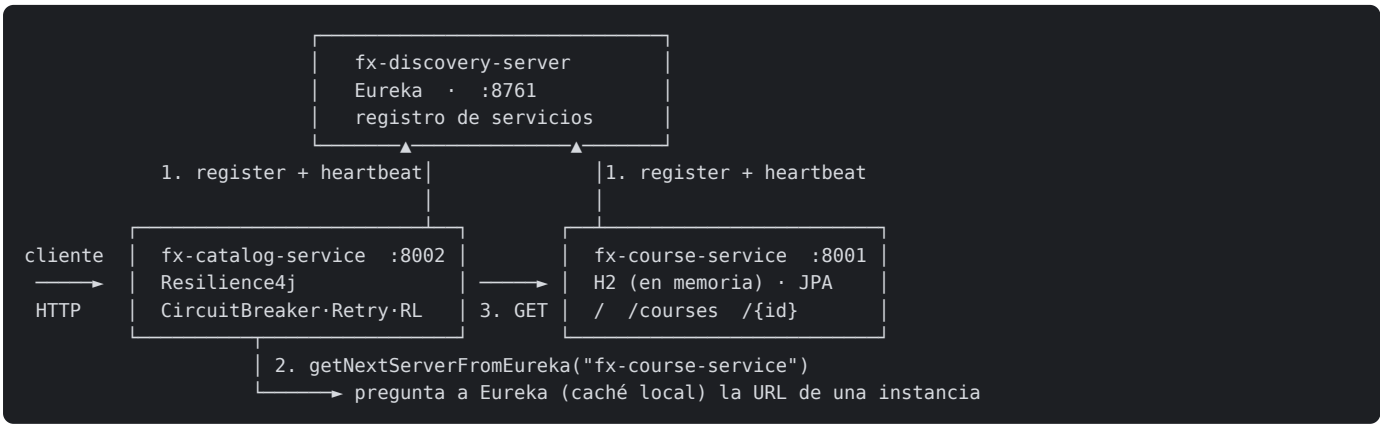
Contenido

1. **Service Discovery** — cómo `fx-catalog-service` descubre y llama a `fx-course-service` (40 %)
2. **Implementación de Resilience4j** — dependencias, propiedades, anotaciones y fallbacks (60 %)
3. **Demostración** — antes y después, apertura y cierre del Circuit Breaker, Retry y RateLimiter
4. Cómo Resilience4j mejora la resiliencia del sistema
5. Cómo Hystrix mejoraba la resiliencia y por qué se reemplazó
6. Cómo reproducir la demostración

> **Entorno:** Java 21 (Temurin) · Maven 3.9 · Spring Boot 3.3.3 · Spring Cloud 2023.0.3. Todo se ejecutó en local: Eureka `:8761`, `fx-course-service` `:8001`, `fx-catalog-service` `:8002`.

1. Service Discovery (40 %)

1.1 Arquitectura



1.2 Cómo se registra un servicio

Cada microservicio incluye `spring-cloud-starter-netflix-eureka-client`. Al arrancar, el cliente toma su nombre de `spring.application.name` y se **registra** en el servidor indicado por `eureka.client.service-url.defaultZone`; después envía un **heartbeat** (renovación de lease) cada 30 s. Si los heartbeats se detienen, Eureka expulsa la instancia.

```
# fx-course-service  (application.properties)
spring.application.name=fx-course-service
server.port=8001
eureka.client.service-url.defaultZone: http://localhost:8761/eureka/

# fx-discovery-server  (application.properties)
server.port=8761
eureka.client.register-with-eureka=false   # el servidor no se registra a sí mismo
eureka.client.fetch-registry=false
```

1.3 Cómo `fx-catalog-service` descubre a `fx-course-service`

El catálogo **no conoce** la dirección del servicio de cursos: solo conoce su **nombre lógico**. En cada petición:

1. `client.getNextServerFromEureka("fx-course-service", false)` consulta el registro de Eureka (copia local del cliente) y devuelve una `InstanceInfo` de una instancia **UP** (round-robin si hay varias).
2. `instanceInfo.getHomePageUrl()` entrega la URL real, por ejemplo `http://172.30.207.195:8001/`.
3. `RestTemplate.getForObject(url + "courses", String.class)` realiza la llamada HTTP al servicio de cursos.

```
private String courseServiceUrl() {
    InstanceInfo instanceInfo = client.getNextServerFromEureka(COURSE_SERVICE_ID, false);   // 1. descubrimiento
    return instanceInfo.getHomePageUrl();                                                   // 2. URL real
}
...
restTemplate.getForObject(courseServiceUrl() + "courses", String.class);                   // 3. llamada
```

Ventaja: si `fx-course-service` cambia de puerto, de máquina o aparecen más instancias, el catálogo no necesita cambios.

1.4 Demostración de la funcionalidad por endpoints

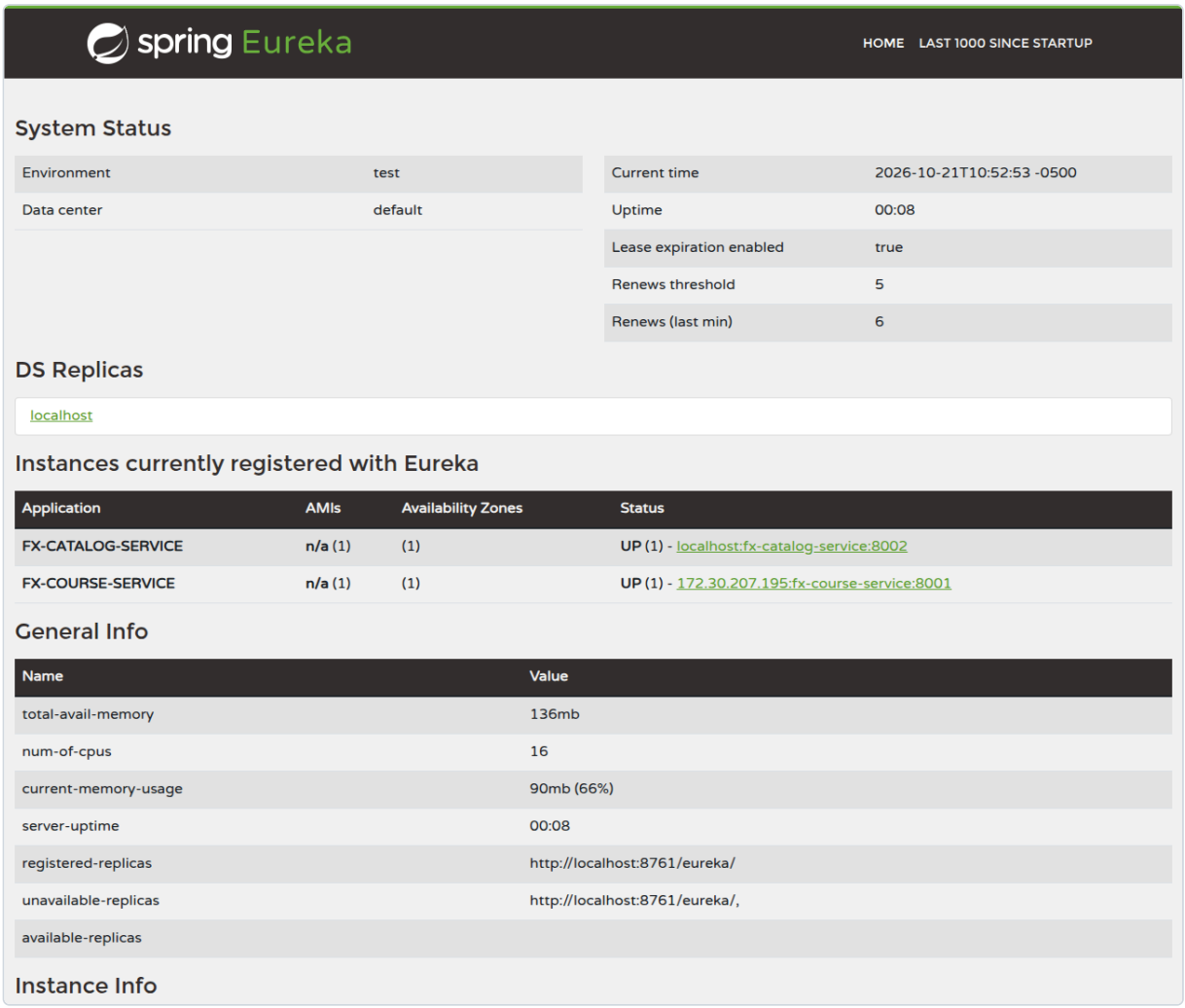


Figura 1. Panel de Eureka (`http://localhost:8761/`): FX-CATALOG-SERVICE (8002) y FX-COURSE-SERVICE (8001) registrados y en estado UP.



Figura 2. `GET http://localhost:8002/` — el catálogo descubre al servicio de cursos y concatena su respuesta.

Our courses are [{"courseid":1,"coursename":"Spring Boot Fundamentals","author":"John Doe"},{"courseid":2,"coursename":"Advanced Java Programming","author":"Jane Smith"},{"courseid":3,"coursename":"Microservices Architecture","author":"Bob Johnson"}]

Figura 3. `GET http://localhost:8002/catalog` — lista de cursos obtenida de `fx-course-service` (H2).



Figura 4. `GET http://localhost:8002/firstcourse` — primer curso obtenido vía descubrimiento.

> **Observación durante las pruebas (caché del registro):** al reiniciar `fx-course-service` el catálogo falló unos segundos con `No matches for the virtual host name :fx-course-service`. Eureka es un registro con **consistencia eventual**: el servidor sirve respuestas cacheadas (~30 s) y el cliente mantiene su copia local. Se redujo en el catálogo con `eureka.client.registry-fetch-interval-seconds: 10`. Esto es justo el tipo de fallo transitorio que el Circuit Breaker debe absorber.

2. Implementación de Resilience4j (60 %)

2.1 Qué se modificó en `fx-catalog-service`

Archivo	Cambio
`pom.xml`	Dependencias `spring-cloud-starter-circuitbreaker-resilience4j`, `spring-boot-starter-aop` (necesaria para que las anotaciones funcionen como aspectos) y `spring-boot-starter-actuator` (monitoreo).
`application.yml`	Reemplaza a `application.properties`. Instancia `courseService` de CircuitBreaker (configuración del taller), Retry y RateLimiter, y endpoints de Actuator.
`CatalogController.java`	Anotaciones `@CircuitBreaker`, `@Retry`, `@RateLimiter` y tres métodos de fallback.
`RestClientConfig.java`	`RestTemplate` con timeouts (conexión 1 s, lectura 2 s): una llamada colgada debe fallar rápido para que el circuito la cuente.

2.2 Dependencias (`pom.xml`)

```
<dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-aop</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
```

2.3 Propiedades (`application.yml`)

```
spring:
  application:
    name: fx-catalog-service

server:
  port: 8002

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
    # El cliente guarda una copia local del registro; la refrescamos cada 10 s (por defecto 30 s)
    registry-fetch-interval-seconds: 10

# --- Resilience4j -------------------------------------------------------
resilience4j:
  circuitbreaker:
    instances:
      courseService:
        registerHealthIndicator: true
        slidingWindowSize: 10
        minimumNumberOfCalls: 5
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
        waitDurationInOpenState: 5s
        failureRateThreshold: 50
        eventConsumerBufferSize: 10
        # Un rechazo del RateLimiter no cuenta como fallo del servicio de cursos
        ignoreExceptions:
          - io.github.resilience4j.ratelimiter.RequestNotPermitted
  retry:
    instances:
      courseService:
        maxAttempts: 3
        waitDuration: 500ms
        eventConsumerBufferSize: 10
        # Con el circuito OPEN no tiene sentido reintentar: se debe responder de inmediato (fail fast)
        ignoreExceptions:
          - io.github.resilience4j.circuitbreaker.CallNotPermittedException
  ratelimiter:
    instances:
      courseService:
        limitForPeriod: 5
        limitRefreshPeriod: 10s
        timeoutDuration: 0

# --- Monitoreo con Actuator ---------------------------------------------
```

```
management:
  health:
    circuitbreakers:
      enabled: true
  endpoints:
    web:
      exposure:
        include: health,circuitbreakers,circuitbreakerevents,retries,retryevents,ratelimiters
  endpoint:
    health:
      show-details: always
```

Propiedad	Valor	Efecto
`slidingWindowSize`	10	El circuito evalúa las últimas 10 llamadas (ventana por conteo).
`minimumNumberOfCalls`	5	No calcula la tasa de fallos hasta tener al menos 5 llamadas registradas.
`failureRateThreshold`	50 %	Si fallan ≥ 50 % de las llamadas de la ventana, el circuito pasa a **OPEN**.
`waitDurationInOpenState`	5 s	Tiempo en OPEN (rechazando llamadas) antes de probar de nuevo.
`automaticTransitionFromOpenToHalfOpenEnabled`	true	Pasa solo a **HALF_OPEN** al cumplirse los 5 s, sin esperar una petición.
`permittedNumberOfCallsInHalfOpenState`	3	En HALF_OPEN deja pasar 3 llamadas de prueba: si el resultado es bueno cierra el circuito; si no, vuelve a OPEN.
`registerHealthIndicator`	true	Publica el estado del circuito en `/actuator/health`.
`eventConsumerBufferSize`	10	Guarda los últimos 10 eventos, visibles en `/actuator/circuitbreakerevents`.
`ignoreExceptions`	`RequestNotPermitted`	Un rechazo del RateLimiter no cuenta como fallo del servicio de cursos.
`retry.maxAttempts / waitDuration`	3 / 500 ms	Reintenta hasta 3 veces con 500 ms entre intentos.
`retry.ignoreExceptions`	`CallNotPermittedException`	Con el circuito OPEN no se reintenta: se responde de inmediato. Sin esto, cada rechazo costaba ~1 s (2 esperas de 500 ms) y los 5 s de OPEN transcurrían antes de lo esperado.
`ratelimiter.limitForPeriod / limitRefreshPeriod`	5 / 10 s	Máximo 5 llamadas cada 10 s; la 6.ª se rechaza de inmediato (`timeoutDuration: 0`).

2.4 Anotaciones y métodos de fallback (`CatalogController`)

```
@GetMapping("/")
    @CircuitBreaker(name = "courseService", fallbackMethod = "fallbackGetCatalogHome")
    public String getCatalogHome() {
        String courseAppMessage = restTemplate.getForObject(courseServiceUrl(), String.class);
        return "Welcome to FutureX Course Catalog " + courseAppMessage;
    }

    // El fallback va en @Retry (la anotación más externa): si estuviera en @CircuitBreaker
    // el error quedaría absorbido y @Retry nunca llegaría a reintentar.
    @GetMapping("/catalog")
    @Retry(name = "courseService", fallbackMethod = "fallbackGetCatalog")
    @CircuitBreaker(name = "courseService")
    public String getCatalog() {
        String courses = restTemplate.getForObject(courseServiceUrl() + "courses", String.class);
        return "Our courses are " + courses;
    }

    @GetMapping("/firstcourse")
    @CircuitBreaker(name = "courseService", fallbackMethod = "fallbackGetSpecificCourse")
```

```
    @RateLimiter(name = "courseService")
    public String getSpecificCourse() {
        Course course = restTemplate.getForObject(courseServiceUrl() + "1", Course.class);
        return "Our first course is " + course.getCoursename();
    }

    // ---- Métodos de fallback (misma firma + el Exception como último parámetro) ----

    public String fallbackGetCatalogHome(Exception ex) {
        log.warn("fallbackGetCatalogHome: {}", ex.toString());
        return "Fallback: Welcome to FutureX Course Catalog, but the course service is not available (" + reason(ex) + ").";
    }

    public String fallbackGetCatalog(Exception ex) {
        log.warn("fallbackGetCatalog: {}", ex.toString());
        return "Fallback: Unable to retrieve course catalog (" + reason(ex) + ").";
    }

    public String fallbackGetSpecificCourse(Exception ex) {
        log.warn("fallbackGetSpecificCourse: {}", ex.toString());
        return "Fallback: Unable to retrieve the first course (" + reason(ex) + ").";
    }

    private String reason(Exception ex) {
        if (ex instanceof CallNotPermittedException) {
            return "circuit breaker is OPEN";
        }
        if (ex instanceof RequestNotPermitted) {
            return "rate limit exceeded";
        }
        return "course service unavailable";
    }
```

- **Fallback:** mismo tipo de retorno (`String`) y mismos parámetros que el método protegido, más un `Exception` final. Se ejecuta cuando la llamada falla *o* cuando el circuito está abierto (`CallNotPermittedException`). El método `reason()` distingue las causas para que la respuesta sea informativa.
- **Orden de los aspectos** de Resilience4j: `Retry ( CircuitBreaker ( RateLimiter ( TimeLimiter ( Bulkhead ( función ) ) ) ) )`. Por eso, en `/catalog` el fallback se declara en `@Retry` (la más externa): un fallback en `@CircuitBreaker` absorbería el error y el Retry nunca reintentaría.
- En `/` y `/firstcourse` el fallback va en `@CircuitBreaker`, como indica la guía del taller.

La clase de configuración opcional `Resilience4JConfig` de la guía (`Customizer<Resilience4JCircuitBreakerFactory>`) configura la API programática de Spring Cloud CircuitBreaker, no las anotaciones; como aquí se usan anotaciones con propiedades en `application.yml`, no hizo falta.

2.5 `RestClientConfig`

```
@Configuration
public class RestClientConfig {

    // Timeouts cortos: una llamada colgada debe fallar rápido para que el Circuit Breaker la cuente.
    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder
                .setConnectTimeout(Duration.ofSeconds(1))
                .setReadTimeout(Duration.ofSeconds(2))
                .build();
    }
}
```

3. Demostración

3.1 Antes: sin Resilience4j

Con `fx-course-service` detenido, el catálogo original propaga el error al cliente:

```
--- GET /catalog (sin Resilience4j, course-service caído)
{"timestamp":"2026-10-21T15:45:56.325+00:00","status":500,"error":"Internal Server Error","path":"/catalog"}
[HTTP 500]
--- GET /firstcourse (sin Resilience4j, course-service caído)
{"timestamp":"2026-10-21T15:45:56.347+00:00","status":500,"error":"Internal Server Error","path":"/firstcourse"}
[HTTP 500]
```

3.2 Después: con Resilience4j (`scripts/demo.sh`)

El script ejecuta cinco escenarios contra `fx-catalog-service` y controla por sí mismo `fx-course-service` (lo detiene y lo reinicia). Todas las respuestas del cliente son **HTTP 200** con un mensaje controlado, en lugar de HTTP 500.

```
=== ESCENARIO 1: todo funcionando (course-service UP, circuito CLOSED) ===
  GET / -> [HTTP 200] Welcome to FutureX Course Catalog Course App Home
  GET /catalog -> [HTTP 200] Our courses are [{"courseid":1,"coursename":"Spring Boot Fundamentals","author":"John Doe"},
{"courseid":2,"coursename":"Advanced Java Progra
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  Circuit Breaker 'courseService': CLOSED | failureRate -1.0% | failedCalls 0 | notPermitted 0

=== ESCENARIO 2: se detiene fx-course-service -> fallback y apertura del circuito ===
  [llamada 1]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (course service unavailable).
  [llamada 2]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (circuit breaker is OPEN).
  [llamada 3]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (circuit breaker is OPEN).
  [llamada 4]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (circuit breaker is OPEN).
  [llamada 5]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (circuit breaker is OPEN).
  [llamada 6]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (circuit breaker is OPEN).
  Circuit Breaker 'courseService': OPEN | failureRate 50.0% | failedCalls 4 | notPermitted 5

=== ESCENARIO 3: circuito OPEN rechaza sin intentar la llamada (fail fast) ===
  GET / -> [HTTP 200] Fallback: Welcome to FutureX Course Catalog, but the course service is not available (circuit breaker is OPEN).
  GET /firstcourse -> [HTTP 200] Fallback: Unable to retrieve the first course (circuit breaker is OPEN).
  Circuit Breaker 'courseService': OPEN | failureRate 50.0% | failedCalls 4 | notPermitted 7

=== ESCENARIO 4: se reinicia fx-course-service; tras 5 s el circuito pasa a HALF_OPEN y luego a CLOSED ===
  (esperando 45 s: Eureka server y la caché local del catálogo deben ver de nuevo a fx-course-service;
   mientras tanto el circuito ya pasó solo de OPEN a HALF_OPEN tras 5 s)
  GET / -> [HTTP 200] Welcome to FutureX Course Catalog Course App Home
  GET /catalog -> [HTTP 200] Our courses are [{"courseid":1,"coursename":"Spring Boot Fundamentals","author":"John Doe"},
{"courseid":2,"coursename":"Advanced Java Progra
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  Circuit Breaker 'courseService': CLOSED | failureRate -1.0% | failedCalls 0 | notPermitted 0

=== ESCENARIO 5: RateLimiter (5 llamadas / 10 s) sobre /firstcourse ===
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  GET /firstcourse -> [HTTP 200] Fallback: Unable to retrieve the first course (rate limit exceeded).
  GET /firstcourse -> [HTTP 200] Fallback: Unable to retrieve the first course (rate limit exceeded).

=== Eventos del Circuit Breaker (actuator/circuitbreakerevents) ===
   11:01:49.320 SUCCESS None
   11:01:49.364 SUCCESS None
   11:01:49.364 STATE_TRANSITION HALF_OPEN_TO_CLOSED
   11:01:59.456 SUCCESS None
   11:01:59.501 SUCCESS None
   11:01:59.536 SUCCESS None
   11:01:59.565 SUCCESS None
   11:01:59.599 SUCCESS None
   11:01:59.625 IGNORED_ERROR None io.github.resilience4j.ratelimiter.RequestNotPermitted: RateLimiter 'c
   11:01:59.649 IGNORED_ERROR None io.github.resilience4j.ratelimiter.RequestNotPermitted: RateLimiter 'c
```

3.3 Qué demuestra cada escenario

Escenario	Observación	Patrón
1. Todo UP	Respuestas normales; circuito **CLOSED**, 0 fallos.	Operación normal
2. Course-service caído	La 1.ª llamada reintenta 3 veces y cae al fallback («course service unavailable»). Al llegar al 50 % de fallos con al menos 5 llamadas registradas el circuito pasa a **OPEN** y los mensajes cambian a «circuit breaker is OPEN».	Fallback + Retry + apertura
3. Circuito OPEN	Las llamadas se rechazan **sin intentar** la conexión (`notPermitted` sube y no hay reintentos): respuesta inmediata y no se satura al servicio caído.	Fail fast
4. Recuperación	Tras 5 s el circuito pasa solo a **HALF_OPEN**; con el servicio de nuevo disponible, las 3 llamadas de prueba tienen éxito y el circuito vuelve a **CLOSED** (evento `HALF_OPEN_TO_CLOSED`).	Auto-recuperación
5. RateLimiter	Las 5 primeras llamadas pasan; la 6.ª y 7.ª se rechazan con «rate limit exceeded» y quedan como `IGNORED_ERROR` (no abren el circuito).	Limitación de tasa

3.4 Retry y monitoreo con Actuator

```
GET /actuator/retryevents  (buffer de 10 eventos)
   11:00:51.110 RETRY intento 1
```

```
11:00:51.614 RETRY intento 2
11:00:52.118 ERROR intento 3
11:00:52.166 RETRY intento 1
11:00:52.668 IGNORED_ERROR intento 0
11:00:52.708 IGNORED_ERROR intento 0
11:00:52.747 IGNORED_ERROR intento 0
11:00:52.780 IGNORED_ERROR intento 0
11:00:52.815 IGNORED_ERROR intento 0
```

La primera petición fallida a `/catalog` produce `RETRY 1`, `RETRY 2` y `ERROR 3` con ~500 ms entre intentos, y entonces se ejecuta el fallback. Cuando el circuito ya está OPEN, los rechazos aparecen como `IGNORED_ERROR` con intervalos de ~40 ms: **no se reintentan** (gracias a `retry.ignoreExceptions`) y la respuesta es inmediata.

```
GET /actuator/health  (componente circuitBreakers)
{
  "status": "UP",
  "details": {
    "courseService": {
      "status": "UP",
      "details": {
        "failureRate": "0.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "0.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 5,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 0,
        "notPermittedCalls": 0,
        "state": "CLOSED"
      }
    }
  }
}
```

4. Cómo Resilience4j mejora la resiliencia del sistema

En una arquitectura de microservicios una llamada remota puede fallar o tardar. Sin protección, un servicio caído provoca **fallos en cascada**: los hilos del llamador quedan esperando, se agotan los recursos y el fallo se propaga hacia arriba. Resilience4j lo evita con patrones componibles:

Patrón	Problema que resuelve	En este proyecto
Circuit Breaker	Deja de llamar a un servicio que está fallando, le da tiempo a recuperarse y evita consumir recursos en llamadas condenadas al fallo.	`@CircuitBreaker(courseService)` en los tres endpoints
Fallback	Ofrece una degradación controlada (respuesta alternativa) en vez de un error 500.	Métodos `fallbackGet*`
Retry	Absorbe fallos transitorios (red, reinicio, caché de Eureka).	`@Retry` en `/catalog`
RateLimiter	Protege al servicio de ráfagas de tráfico.	`@RateLimiter` en `/firstcourse`
Bulkhead / TimeLimiter	Aíslan recursos por dependencia y acotan la duración de una llamada.	No aplicados; el timeout se resolvió con `RestTemplate`

Resultado medido: sin Resilience4j la caída de `fx-course-service` producía **HTTP 500** en el catálogo; con Resilience4j el catálogo sigue disponible con respuestas degradadas, el circuito protege al servicio caído mientras está OPEN y el sistema **se recupera solo** cuando el servicio vuelve.

5. Cómo Hystrix mejoraba la resiliencia (y por qué se reemplaza)

Hystrix (Netflix) fue la primera librería de tolerancia a fallos ampliamente usada con Spring Cloud. Mejoraba la resiliencia envolviendo cada llamada remota en un `HystrixCommand` (patrón Command) y aplicando:

- **Circuit Breaker** sobre una ventana de tiempo deslizante (por defecto 10 s): abre el circuito si se supera un porcentaje de fallos y rechaza llamadas durante un periodo.
- **Aislamiento por *bulkhead***: cada dependencia se ejecuta en su propio *thread pool* (o con semáforos), de modo que un servicio lento no agota los hilos de toda la aplicación.
- **Timeouts** por comando y **fallback** (`@HystrixCommand(fallbackMethod=...)`).
- **Request collapsing/caching** y un **Hystrix Dashboard** con Turbine para ver el estado de los circuitos.

Hystrix entró en **modo mantenimiento en 2018** y Spring Cloud lo eliminó desde la versión 2020.0; para Spring Boot 3 se usa Spring Cloud CircuitBreaker con Resilience4j.

Aspecto	Hystrix	Resilience4j
Estado	Mantenimiento (sin evolución)	Activo; soportado por Spring Boot 3
Diseño	Objeto `HystrixCommand`, dependencias de Netflix (Archaius, RxJava 1)	Decoradores funcionales (Java 8+), librería ligera y modular
Aislamiento	Thread pools por defecto (más sobrecarga)	Semáforos (`Bulkhead`); thread pool opcional
Ventana de fallos	Por tiempo	Por conteo o por tiempo
Patrones	CircuitBreaker, fallback, aislamiento, collapsing	CircuitBreaker, Retry, RateLimiter, Bulkhead, TimeLimiter, Cache
Estados	Cerrado / abierto / semiabierto	CLOSED / OPEN / HALF_OPEN (+ DISABLED, FORCED_OPEN)
Métricas	Hystrix Dashboard + Turbine	Micrometer + Actuator (`/actuator/circuitbreakers`)
Configuración	Propiedades `hystrix.command.*`	`resilience4j.*` en `application.yml` o anotaciones

6. Cómo reproducir

```
# 1) Clonar y compilar los tres proyectos (Java 17+, Maven)
git clone https://github.com/jrquinte/FutureXEurekaServer  fx-discovery-server
git clone https://github.com/jrquinte/FutureXCourseApp      fx-course-service
git clone https://github.com/jrquinte/FutureXCourseCatalog  fx-catalog-service   # + rama feature/resilience4j

# 2) Orden de arranque: Eureka (8761) -> course (8001) -> catalog (8002)
./scripts/run-all.sh

# 3) Probar
curl http://localhost:8002/catalog
curl http://localhost:8002/actuator/circuitbreakers

# 4) Demostración completa (detiene y reinicia fx-course-service; ~3 min)
./scripts/demo.sh
```